
Loop alignment: Self-organized Weight Transpose in Predictive Coding through Independent Hebbian Plasticity.

Anonymous Author(s)

Affiliation

Address

email

Abstract

Backpropagation (BP) requires feedback (FB) weights to equal the transpose of feedforward (FF) weights at every layer and at every step. This issue is called the weight transport problem, and has been considered to be biologically implausible. Neural circuits are not built from anatomically paired FF/FB axons but from asymmetric projections that form recurrent loops shared across many computations. In multi-nuclei circuits (e.g., thalamo-cortical, cortico-cerebellar, basal-ganglia loops), the return path traverses entirely distinct nuclear structures, making any mechanism for maintaining synchronized weight transposes anatomically absent. Predictive coding (PC), originally proposed as a biologically plausible substitute for BP, silently inherits this constraint despite its biological motivation. Therefore, we propose two PC variants that eliminate it entirely. **PC-DH** (Dual Hebbian plasticity model) updates FF and FB weights by independent local Hebbian rules with no inter-pathway coordination. **PC-RFB** (Random FeedBack) fixes the FB weight with a random matrix, testing the robustness of the learning mechanism. Both match baseline PC accuracy ($\approx 85\%$ on MNIST). We identify a phenomenon we termed **Loop Alignment**: the spontaneous convergence of FF and FB weights toward a transpose relationship, driven by the structure of the inference loop. We then prove that it holds exactly at every training step in both models. To test whether Loop Alignment holds in the multi-nuclei setting, we conduct a circuit-sharing experiment: two networks learn different tasks (Net 1: MNIST; Net 2: Fashion-MNIST) over a shared pathway, where each network's FB is routed through the other's FF weights. Even when the entire shared pathway is plastic and driven simultaneously by both tasks, each network solves its task independently and FF/FB alignment is stronger than in the isolated case. Loop Alignment explains how the brain can learn effectively without weight transport, and support multi-task learning over shared asymmetric circuits.

1 Introduction

Neural circuits are organized as recurrent loops, not as anatomically paired feedforward (FF) and feedback (FB) channels. In cortico-cortical projections, FF and FB axons originate from different cell types, occupy distinct laminar compartments, and exhibit plasticity rules that differ in form, magnitude, and time scale [Thomson and Bannister, 2003, Bastos et al., 2012, Gerstner et al., 2014]. The asymmetry runs deeper in multi-nuclei circuits, for example, in thalamo-cortical, cortico-cerebellar, and basal-ganglia loops, the return path traverses entirely distinct nuclear structures populated by separate cell populations with no shared plasticity signal. In both cases, what is shared is the anatomical channel—a bundle of asymmetric projections connecting two populations—not a co-

ordinated, functionally paired set of axons. The same pathway participates in perception, attention, motor prediction, memory retrieval, and context modulation, often simultaneously and in different roles [Rigotti et al., 2013]. There is no strict anatomical pairing of individual FF and FB axons into coordinated monosynaptic loops.

This biological reality places the weight transport problem at the center of any biologically plausible learning theory. Backpropagation (BP) [Rumelhart et al., 1986] requires the FB weight matrix at every layer to be the exact transpose of the corresponding FF matrix at every training step. In a brain where bidirectional pathways traverse multiple relay nuclei, engage separate cell populations, and serve overlapping functional roles, no mechanism exists to enforce—or even plausibly approximate—this synchronized transpose relationship. The weight transport constraint is not merely unverified; it is structurally incompatible with the known anatomy of both cortical and subcortical circuits.

Predictive coding (PC) has emerged as the leading biologically plausible alternative to BP [Rao and Ballard, 1999, Friston, 2005, 2010, Whittington and Bogacz, 2017]. In PC, each cortical layer generates top-down predictions; discrepancies are encoded by local error neurons; and both representation and error activities are iteratively refined through bidirectional message passing before any weight update occurs. Whittington and Bogacz [2017] showed that PC weight updates at inference equilibrium approximate BP gradients, making PC a principled route to gradient-based learning without global error broadcast. Yet PC inherits the weight transport problem: its standard formulation requires $W_i^{\text{FB}} = (W_i^{\text{FF}})^{\top}$ at every layer and every training step—precisely the constraint that cortical and subcortical anatomy makes implausible.

We therefore asked whether the weight transport problem can be dissolved by PC’s own inference loop dynamics, without any explicit coordination between FF and FB pathways. The key insight, which emerges from building our models, is that PC’s iterative inference loop is itself an implicit weight-alignment engine: weight symmetry is an attractor that inference dynamics naturally reach, not a constraint to be imposed from outside.

Approach. Our primary model, **PC-DH** (Dual Hebbian Plasticity), updates FF and FB weights by fully independent local Hebbian rules. No information is shared between the two update steps. We prove that the FF and FB updates are exact transposes of one another at every step, so the two weight matrices converge to a transpose relationship as the contribution of the random initialization is washed out.

To isolate the inference loop’s contribution from any FB learning rule, we introduce **PC-RFB**: the FB pathway is a fixed random matrix, never updated. This removes even the possibility of coordinated adaptation between pathways, capturing the extreme case in which FF and FB axons belong to entirely separate anatomical systems. Even under this maximally constrained condition, FF weights spontaneously align with the random FB structure (cosine similarity ≈ 0.9). We derive the explicit inference-driven drift term responsible for this alignment, showing that the inference loop alone without any FB plasticity is sufficient to drive FF weights toward an approximate transpose relationship.

Circuit multiplexing as functional validation. If FB pathways need not carry precisely calibrated, transpose-matched signals, they can be shared, repurposed, and overlapping—as multi-nuclei anatomy directly implies. We test this by constructing a circuit-sharing scenario that instantiates this topology: two PC networks (Net 1 learning MNIST; Net 2 learning Fashion-MNIST) share a single bidirectional pathway, with each network’s FB signal routed through the other network’s continuously learned FF weights. This produces an extreme entanglement: each network’s FB pathway is reshaped at every step by the other network’s learning on a different task. Loop Alignment persists under this condition—both networks reach $\approx 85\%$ accuracy on their respective tasks while maintaining strong FF–FB cosine similarity. In the fully plastic variant, where every weight in the shared loop is updated simultaneously by both networks’ Hebbian rules, alignment quality is *higher* than in the single-task isolated case, suggesting that global plasticity actively supports rather than threatens Loop Alignment.

Contributions.

1. **PC-DH and an exact Loop Alignment result.** We propose PC-DH model and prove that two independently updated local Hebbian rules produce cumulative updates that are exact transposes of each other at every step (Section 4.1).
2. **PC-RFB and the inference-driven drift mechanism.** We introduce PC-RFB and derive the explicit drift term, generated by the inference loop, that drives FF weights toward the random FB scaffold without any FB plasticity (Section 3.3).
3. **Multi-task circuit sharing on a fully plastic shared loop.** We demonstrate that two networks sharing bidirectional pathways, and solving distinct tasks maintain Loop Alignment, with performance and alignment quality strengthened when all shared weights are simultaneously learnable (Section 5.3).

2 Related work

Predictive coding. Rao and Ballard [1999] introduced hierarchical PC as a model of visual cortex. The free-energy principle [Friston, 2005, 2010, Friston and Kiebel, 2009] generalized PC to a unified theory of perception and learning. Bastos et al. [2012] provided neurophysiological evidence mapping superficial cortical layers to bottom-up error signals and deep layers to top-down predictions. Whittington and Bogacz [2017] proved that PC’s local Hebbian updates, at convergence, recover BP gradients. Millidge et al. [2022] review the field. All prior formulations assume $W_i^{\text{FB}} = (W_i^{\text{FF}})^{\top}$; weight transport has been acknowledged but not resolved within PC. A complementary challenge in scaling PC networks is depth degradation: performance drops sharply beyond five to seven layers due to exponentially imbalanced prediction errors across layers [Qi et al., 2025]. While the present work focuses on the weight transport problem in shallow-to-moderate architectures, combining loop alignment with precision-weighted updates [Qi et al., 2025] is a natural direction for extending biologically plausible learning to deep architectures.

Feedback alignment. Lillicrap et al. [2016] demonstrated that in BP, fixed random FB weights suffice for learning because FF weights align with the random FB matrix over training (Feedback Alignment, FA). Extensions include direct FA [Nøkland, 2016] and sign-symmetry studies [Liao et al., 2016]. These operate within BP’s single-pass computational graph. In PC-DH, FF and FB weights are learned by independent Hebbian rules but self-organize toward a transpose relationship through PC’s bidirectional inference dynamics. PC-RFB is inspired by the FA, but we find that the alignment mechanism is qualitatively different from original FA.

Equilibrium propagation, target propagation, and forward-only learning. Equilibrium propagation [Scellier and Bengio, 2017] computes gradients through energy perturbations. Recent work [Laborieux and Zenke, 2024] addressed its analogous weight-symmetry requirement via Jacobian homeostasis. Contrastive Hebbian learning [Hinton, 2002], target propagation [Lee et al., 2015], dendritic microcircuits [Guerguiev et al., 2017, Sacramento et al., 2018], and forward-forward [Hinton, 2022] employ local objectives but require distinct computational phases or auxiliary structures, while our models operate within the standard single-phase PC inference loop.

3 Models

We use the supervised PC model of Whittington and Bogacz [2017] as our baseline. The network has three layers with sizes 784–100–10 for representation neurons and two layers of error neurons (sizes 100 and 10). We omit nonlinear activations, adaptive optimizers, and normalization from this study, ensuring that any observed alignment is attributable to the core PC dynamics and Hebbian plasticity, not to auxiliary deep-learning machinery.

131 3.1 Baseline predictive coding

132 At each layer i :

$$\varepsilon_i = \mathbf{x}_{i+1} - \mathbf{x}_i W_i^{\text{FF}}, \quad (1)$$

$$\Delta \mathbf{x}_i = \eta(-\varepsilon_{i-1} + \varepsilon_i W_i^{\text{FB}}), \quad (2)$$

$$\Delta W_i^{\text{FF}} = \alpha(\mathbf{x}_i^\top \varepsilon_i), \quad (3)$$

$$W_i^{\text{FB}} = (W_i^{\text{FF}})^\top. \quad (4)$$

133 For each input, 50 inference loops update $\mathbf{x}_i$ with weights fixed; weights are then updated via Eq. (3).

134 3.2 PC-DH: Predictive Coding with Dual Hebbian Plasticity

135 PC-DH is our primary model. FF and FB weights are independent matrices, each governed by its
136 own local Hebbian rule with no information shared between them. After inference with Eq. (1) and
137 Eq. (2), both weight matrices update independently as follows:

$$\Delta W_i^{\text{FF}} = \alpha(\mathbf{x}_i^\top \varepsilon_i), \quad (5)$$

$$\Delta W_i^{\text{FB}} = \alpha(\varepsilon_i^\top \mathbf{x}_i). \quad (6)$$

138 Eq. (6) contains no reference to W_i^{FF} . No transpose operation, weight copying, or synchronized
139 update is required at any point. Both updates use the same neural activities $\mathbf{x}_i$ and ε_i produced by
140 the same inference loop.

141 3.3 PC-RFB: Predictive Coding with fixed Random FB

142 In PC-RFB, W_i^{FB} is replaced by a fixed random matrix B_i , drawn Xavier-normal at initialization
143 and never updated:

$$\Delta \mathbf{x}_i = \eta(-\varepsilon_{i-1} + \varepsilon_i B_i). \quad (7)$$

144 FF weights are updated as in Eq. (5). The random matrix B_i models the scenario in which the
145 FB pathway traverses relay nuclei with no-plasticity or no task-relevant structure: if the effective
146 FB connectivity is a product of random matrices across nuclei, the result is itself a random matrix.
147 Crucially, FF and FB do not share weight, so there is no monosynaptic reciprocal connection. PC-
148 RFB is meant to isolate the contribution of the inference loop by removing FB learning.

149 4 Theoretical analysis

150 4.1 Loop Alignment is structurally guaranteed in PC-DH

151 The key observation is that the FF and FB Hebbian rules in PC-DH use the same neural activities at
152 every step. Accumulating updates across T training steps,

$$W^{\text{FF}}(T) = W^{\text{FF}}(0) + \alpha \sum_{t=1}^T \mathbf{x}_i(t)^\top \varepsilon_i(t), \quad (8)$$

$$W^{\text{FB}}(T) = W^{\text{FB}}(0) + \alpha \sum_{t=1}^T \varepsilon_i(t)^\top \mathbf{x}_i(t). \quad (9)$$

153 At every single step, the FF and FB outer products are exact transposes of one another:

$$\varepsilon_i(t)^\top \mathbf{x}_i(t) = (\mathbf{x}_i(t)^\top \varepsilon_i(t))^\top. \quad (10)$$

154 This identity holds pointwise, so it distributes over the entire sum:

$$W^{\text{FF}}(T) - W^{\text{FF}}(0) = (W^{\text{FB}}(T) - W^{\text{FB}}(0))^\top. \quad (11)$$

155 The accumulated updates are therefore always in exact transpose. The only obstacle to $W^{\text{FF}}(T) \approx$
156 $(W^{\text{FB}}(T))^\top$ is the initial weights $W^{\text{FF}}(0)$ and $W^{\text{FB}}(0)$. As training progresses, these random
157 initializations are washed out by the growing Hebbian updates.

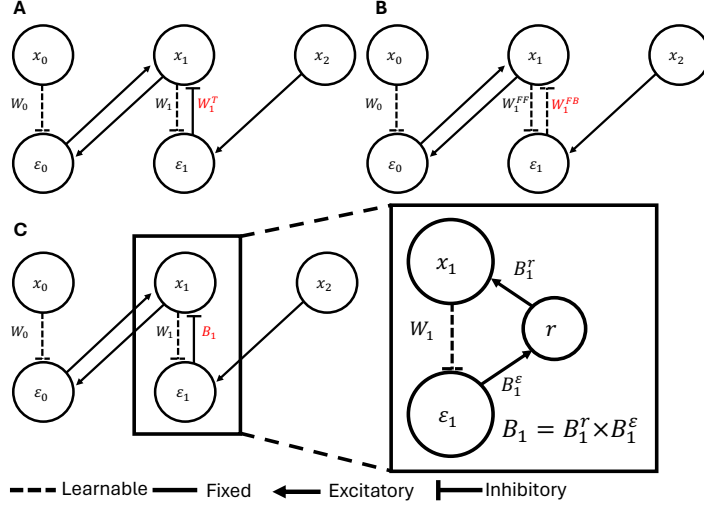


Figure 1: **Model architectures.** Circles: neural populations (x : representation neurons; ε : error neurons). Dashed arrows: learnable weights; solid arrows: fixed weights. **(A)** Baseline PC: FB weights are the transpose of FF weights, encoding the weight transport problem. **(B)** PC-DH: FF and FB are independent matrices updated by fully independent Hebbian rules; no coordination. **(C)** PC-RFB: FB is a fixed random matrix B , modeling FB pathways through asymmetric relay nuclei (right inset: an effective B as the product of random matrices through a relay).

Inference update rule. Under predictive-coding inference dynamics, the representation x_i is adjusted at each inference step to simultaneously reduce the local prediction error ε_{i-1} (the error for which layer i is responsible) and incorporate the top-down feedback signal $\varepsilon_i B_i$ arriving via the fixed random scaffold. A single discrete inference step therefore reads

$$\tilde{x}_i = x_i - \eta \varepsilon_{i-1} + \eta \varepsilon_i B_i. \quad (12)$$

The first correction pulls x_i toward the FF prediction from below; the second incorporates the feedback-weighted error from above. Together they define the inference trajectory whose fixed point is the posterior estimate of x_i given both neighbouring layers.

Deriving the three-term decomposition. After the inference step, the FF Hebbian update is applied using the updated representation $\tilde{x}_i$ as the pre-synaptic term:

$$\Delta W^{FF} = \alpha (\tilde{x}_i^\top \varepsilon_i). \quad (13)$$

Substituting (12) into (13) and expanding linearly:

$$\begin{aligned} \Delta W^{FF} &= \alpha (x_i - \eta \varepsilon_{i-1} + \eta \varepsilon_i B_i)^\top \varepsilon_i \\ &= \alpha x_i^\top \varepsilon_i - \alpha \eta \varepsilon_{i-1}^\top \varepsilon_i + \alpha \eta (\varepsilon_i B_i)^\top \varepsilon_i. \end{aligned} \quad (14)$$

The third term can be simplified as follows:

$$(\varepsilon_i B_i)^\top \varepsilon_i = B_i^\top (\varepsilon_i^\top \varepsilon_i) \quad (15)$$

Substituting back into (14) and we obtain:

$$\Delta W^{FF} = \underbrace{\alpha x_i^\top \varepsilon_i}_{\text{Hebbian}} - \underbrace{\alpha \eta \varepsilon_{i-1}^\top \varepsilon_i}_{\text{cross-layer decorrelation}} + \underbrace{\alpha \eta B_i^\top (\varepsilon_i^\top \varepsilon_i)}_{\text{alignment drift}}. \quad (16)$$

In the PC-DH model, the same decomposition holds with $B_i = W^{FB}$, so the alignment drift term becomes $\alpha \eta W^{FB\top} (\varepsilon_i^\top \varepsilon_i)$ where W^{FF} is nudged toward $W^{FB\top}$ by inference, while the symmetric Hebbian rule simultaneously nudges W^{FB} toward $W^{FF\top}$, driving Loop Alignment from both directions.

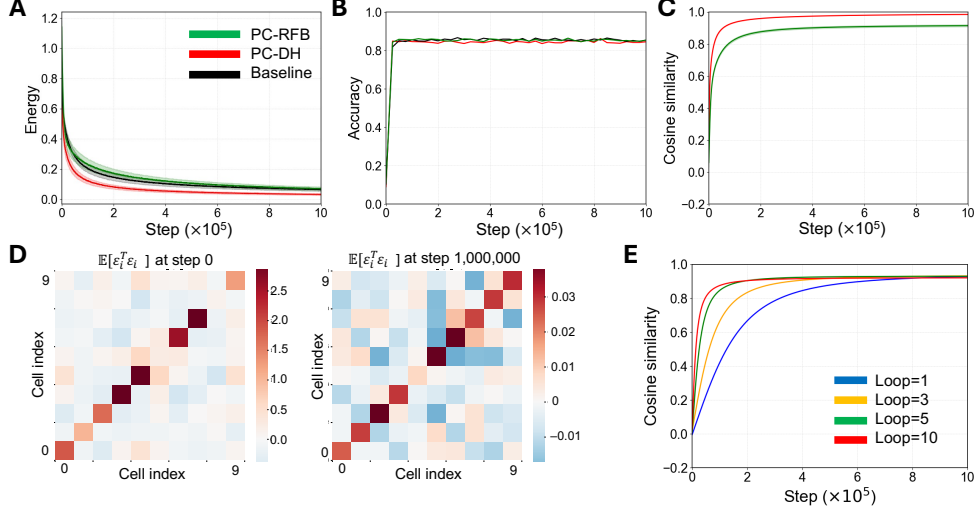


Figure 2: **Single-task learning performance and weight alignment.** Green: PC-RFB; red: PC-DH; black: baseline. Mean over 50 sessions (10^6 steps); shading ± 1 SD. (A) Energy. Both proposed models minimize energy as stably as baseline. (B) MNIST test accuracy. PC-DH and PC-RFB reach $\approx 85\%$, matching baseline (McNemar, $p > 0.05$). (C) FF–FB cosine similarity. PC-DH $\rightarrow 1.0$; PC-RFB $\rightarrow 0.9$. (D) Batch-averaged error auto-correlation $\mathbb{E}[\varepsilon_i^\top \varepsilon_i]$ at start (left) and at convergence (right) of the training in PC-RFB. (E) Effect of inference loop count ($L \in \{1, 3, 5, 10\}$) on FF–FB cosine similarity in PC-RFB.

5 Experiments

Setup. All single-network experiments use a 3-layer architecture (784–100–10), Xavier initialization, mini-batches of 20, and 50 inference loops per input. Main experiments: 10^6 steps, 50 sessions; circuit-sharing and robustness experiments: 3×10^6 steps, 10 sessions. No nonlinear activations, adaptive optimizers, or normalization. The plots show the mean for all sessions (solid line) ± 1 SD (shaded). Full hyperparameters are provided in Appendix A.

5.1 Single-task learning performance

Both PC-DH and PC-RFB achieve monotonic energy minimization and reach $\approx 85\%$ classification accuracy, matching the transpose baseline (Figure 2A–B). Neither model differs significantly from baseline by McNemar’s test (Appendix C).

5.2 Weight alignment dynamics

PC-DH. Cosine similarity between FF and FB weights rises from ≈ 0 (random initialization) to ≈ 1.0 over training (Figure 2C). Two independently updated Hebbian matrices converge to a transpose relationship with no explicit coordination.

PC-RFB. Cosine similarity rises from ≈ 0 to ≈ 0.9 (Figure 2C), showing strong partial alignment.

Single-sample outer products exhibit pronounced off-diagonal structure (rank-1 by construction). However, batch-averaged matrices are diagonally dominant at both stages (Figure 2D), confirming that the whiteness approximation underlying Eq. (25) holds throughout learning. We then examine how loop number can affect those alignment. Figure 2E shows FF–FB cosine similarity for PC-RFB under $L \in \{1, 3, 5, 10\}$ inference loops per step. Alignment occurs under all conditions and increases monotonically with L , consistent with the prediction that each inference loop contributes one application of the alignment drift term.

196 5.3 Multi-task circuit sharing

197 **Circuit structure.** To test our proposed models under biologically realistic circuit entanglement,
 198 we construct a multi-task circuit-sharing setup (Fig. 3A). Two PC networks, Net 1 (MNIST) and
 199 Net 2 (Fashion-MNIST), each maintain their own FF weight matrices $W_i^{\text{Net 1}}$ and $W_i^{\text{Net 2}}$, updated
 200 by independent local Hebbian rules. The two networks share a single bidirectional pathway at
 201 the second layer: the representation/error neurons $\mathbf{x}_i^{\text{Net 1}}, \epsilon_i^{\text{Net 1}}, \mathbf{x}_i^{\text{Net 2}}, \epsilon_i^{\text{Net 2}}$ form a closed loop
 202 in which each network’s FB signal is routed through the *other* network’s FF weights, sandwiched
 203 by two relay matrices $\mathbf{U}$ and $\mathbf{V}$ (Xavier-normal, shared across networks). Biologically, $\mathbf{U}$ can be
 204 thought of as the projection from a network’s local error population into a shared relay nucleus,
 205 while $\mathbf{V}$ returns the processed signal back to the originating network’s representation neurons—an
 206 abstraction of the input/output projections of any multi-nuclei relay (e.g. thalamus in a thalamo-
 207 cortical loop). Because the loop is closed, the two FB composites traverse the loop in opposite
 208 directions:

$$W_i^{\text{FB, Net 1}} = \mathbf{U} W_i^{\text{Net 2}} \mathbf{V}, \quad W_i^{\text{FB, Net 2}} = \mathbf{V} W_i^{\text{Net 1}} \mathbf{U}. \quad (17)$$

209 **Alternating training with passive relay.** The two networks are trained in strict *alternation*: each
 210 training step is dedicated to a single network. We do not use mini-batching; one image is processed
 211 per step. At step t , if the active network is Net 1 we draw $s^{\text{Net 1}}(t) \sim \mathcal{D}_{\text{MNIST}}$ (image and one-hot
 212 label) and clamp Net 1’s bottom and top representations; only Net 1 receives an external input. We
 213 then run an inference phase of $L_{\text{inf}} = 50$ loops, in which Net 1’s $\mathbf{x}^{\text{Net 1}}, \epsilon^{\text{Net 1}}$ are updated by

$$\Delta \mathbf{x}_i^{\text{Net 1}} = \eta (-\epsilon_{i-1}^{\text{Net 1}} + \epsilon_i^{\text{Net 1}} \mathbf{U} W_i^{\text{Net 2}} \mathbf{V}). \quad (18)$$

214 Net 2’s shared-layer activities are not driven by any input during this step; instead, they are computed
 215 *passively as relay values* on Net 1’s signal path,

$$\mathbf{x}_i^{\text{Net 2}} = \epsilon_i^{\text{Net 1}} \mathbf{U}, \quad \epsilon_i^{\text{Net 2}} = \mathbf{x}_i^{\text{Net 2}} W_i^{\text{Net 2}} = \epsilon_i^{\text{Net 1}} \mathbf{U} W_i^{\text{Net 2}}, \quad (19)$$

216 i.e. as the post- $\mathbf{U}$ and post- $W^{\text{Net 2}}$ stages of the loop traversal $\epsilon^{\text{Net 1}} \rightarrow \epsilon^{\text{Net 1}} \mathbf{U} \rightarrow$
 217 $\epsilon^{\text{Net 1}} \mathbf{U} W^{\text{Net 2}} \rightarrow \epsilon^{\text{Net 1}} \mathbf{U} W^{\text{Net 2}} \mathbf{V}$. After the 50 loops complete, the converged values of
 218 $\mathbf{x}^{\text{Net 1}}, \epsilon^{\text{Net 1}}$ together with the relay values $\mathbf{x}^{\text{Net 2}}, \epsilon^{\text{Net 2}}$ are used to update the learnable weights of
 219 this step (only Net 1’s FF in the partial-plasticity case; Net 1’s FF together with $\mathbf{U}$ and $\mathbf{V}$ in the fully
 220 plastic case; full update equations in Appendix E). The next step is symmetric with Net 2 active and
 221 Net 1 serving as the relay; Net 1 and Net 2 use independently shuffled data pools (different random
 222 seeds). The procedure is repeated for 3×10^6 steps.

223 **Partial plasticity (Fig. 3B–D).** $\mathbf{U}$ and $\mathbf{V}$ are fixed throughout training. The only learnable weights
 224 are the task-specific FF matrices, and only the FF of the active network is updated at each step
 225 (Eq. (5)). Despite each network’s effective FB being continuously reshaped by the partner network’s
 226 task-driven learning, both networks successfully minimize energy and learn their tasks (Fig. 3B–C),
 227 with Net 1 (MNIST) and Net 2 (Fashion-MNIST) each reaching $\approx 85\%$ accuracy. Loop Alignment
 228 is preserved: the cosine similarity between $W_i^{\text{Net 1}}$ and its effective feedback weight $\mathbf{U} W_i^{\text{Net 2}} \mathbf{V}$
 229 converges to ≈ 0.9 (Fig. 3D).

230 **Full plasticity (Fig. 3E–G).** On the active network’s step, we additionally apply local Hebbian
 231 updates at the input ($\mathbf{U}$) and output ($\mathbf{V}$) synapses of the relay, using only the active network’s
 232 variables and the relay values defined by Eq. (19) (full update equations in Appendix E). The partner
 233 network’s FF weights are not updated on this step. Both networks learn their tasks without difficulty
 234 (Fig. 3E,F). Interestingly, the cosine similarity converges to an even higher value than in the partial-
 235 plasticity setting (Fig. 3G), suggesting stronger Loop Alignment in fully plastic circuit sharing.

236 5.4 Robustness test with random noise injection

237 To test the robustness of Loop Alignment, we added uniform noise to every element of the FB matrix
 238 B_i at each training step, with amplitude $\delta \in \{1\%, 2\%, 3\%, 5\%, 10\%, 20\%, 30\%\}$ of the Xavier scale.
 239 With noise injection, the weight quickly drift from the initial weight (Fig. 4A). For $\delta \leq 5\%$, accuracy
 240 remains $\approx 85\%$ and cosine similarity converges to 0.2–0.4 (Fig. 4B–D); at $\delta = 10\%$ accuracy
 241 decreases modestly; at $\delta \geq 20\%$, alignment is largely prevented and accuracy degrades substantially.
 242 Notably, learning succeeds at cosine similarities as low as 0.2, showing that exact alignment was not
 243 required for effective PC for the MNIST task.

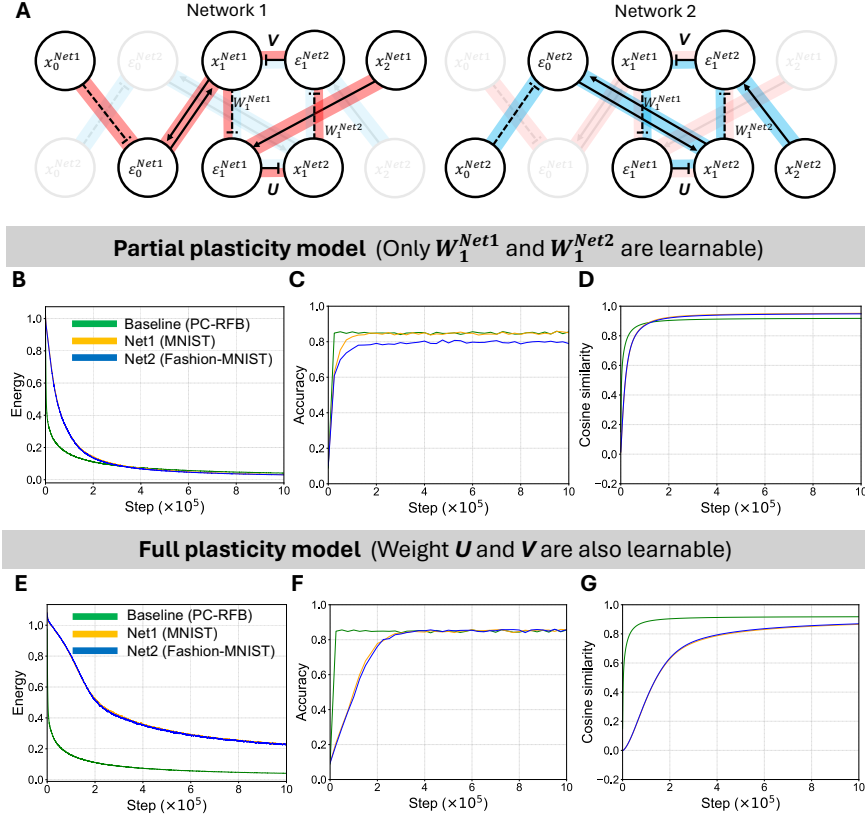


Figure 3: **Multi-task circuit sharing.** Net 1 (MNIST) and Net 2 (Fashion-MNIST) share a bi-directional pathway. Yellow/blue: Net 1/Net 2 in circuit sharing task; green: Baseline with PC-RFB model without circuit share. Mean over 10 sessions (3×10^6 steps); shading ± 1 SD. (A) Overview of circuit structure. (B) Energy of the circuit through learning. (C) Test accuracy on each task. Both networks reach $\approx 85\%$ despite sharing a continuously changing FB pathway. (D) FF-FB cosine similarity. Loop Alignment converges in both networks. (E–G) Fully plastic variant: energy, accuracy, and FF-FB cosine similarity when every shared-pathway weight is simultaneously updated by both tasks.

244 6 Discussion

245 **The inference loop as a weight-alignment mechanism.** The central result of this paper is that
 246 PC’s inference loop is a self-organizing weight-symmetry mechanism. In PC-DH, this is a structural
 247 guarantee because FF and FB Hebbian updates use the same neural activities at the same step, their
 248 cumulative sums are exactly transposes of one another (Eq. 10). In PC-RFB, the inference loop
 249 iteratively adapts neural activity to the FB weight before each weight update, generating the drift
 250 term (Eq. 16; full derivation in Eq. 25) that orients FF weights toward the FB transpose.

251 **Circuit multiplexing and brain theory.** Circuit-sharing provides insight into how the brain sup-
 252 ports multiple overlapping computations across shared anatomical pathways without destructive in-
 253 terference. Loop Alignment does not require dedicated, segregated circuits, rather, independent
 254 Hebbian plasticity in both FF and FB directions is sufficient to maintain weight coherence even
 255 when the shared pathway is continuously reshaped by a different task. A key contributor could
 256 be the cross-layer decorrelation term ($-\alpha \eta \varepsilon_{i-1}^\top \varepsilon_i$) in the weight update (Eq. 16): by subtracting
 257 correlations between adjacent error layers it suppresses interference, preventing each network’s FF
 258 weights from drifting toward the other task’s error directions even as both tasks drive the same shared
 259 weights.

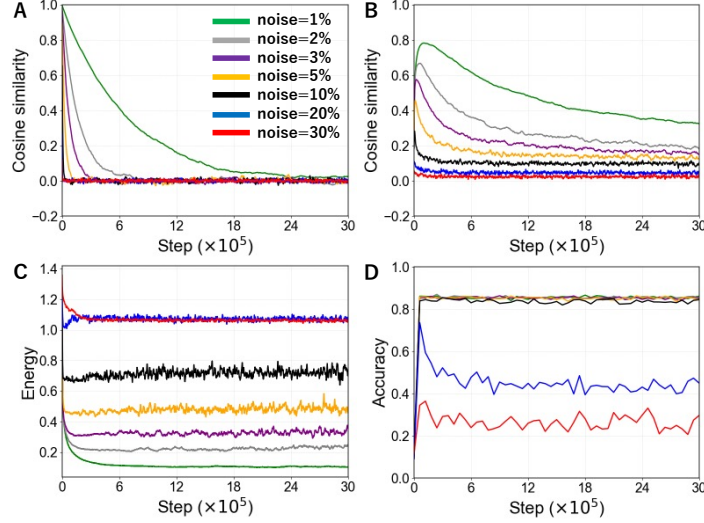


Figure 4: **Robustness test with dynamic FB noise.** PC-RFB model was used to train MNIST with noise injection during the inference loop. **(A)** Cosine similarity between the initial and final FF weights at each noise level, measuring how far the FF weights drifted from initial weight over training. **(B)** FF-FB cosine similarity under continuous additive FB noise ($\delta = 1\%$ – 30% of Xavier scale). Alignment is stable for $\delta < 5\%$ and largely prevented at $\delta \geq 20\%$. **(C)** Learning energy across training steps at each noise level. Energy minimization proceeds normally for low noise and degrades progressively at higher intensities. **(D)** MNIST test accuracy under each noise level. Accuracy is maintained at $\approx 85\%$ for $\delta \leq 5\%$; modest decrease at $\delta = 10\%$; substantial degradation at $\delta \geq 20\%$.

260 **Random FB does not claim that cortex is random.** The frozen random W^{FB} in PC-RFB is a
 261 worst-case scenario, not a model of cortical anatomy. Cortico-cortical projections have layer-specific
 262 termination patterns and cell-type-specific roles [Thomson and Bannister, 2003, Bastos et al., 2012].
 263 The point of PC-RFB is that even at this worst case, learning still works and alignment still emerges;
 264 in real cortex, where FB pathways carry useful top-down signals, alignment should be at least as
 265 effective. Similarly, our use of fully connected matrices is a first-order approximation, therefore
 266 verifying Loop Alignment in sparse, anatomically structured connectivity is an important next step.

267 **Limitations and future directions.** We acknowledge that our networks reach $\approx 85\%$ on MNIST,
 268 well below modern benchmarks. However, by excluding optimizers, ReLU, batch normalization,
 269 and other deep-learning machinery, we ensured that the alignment phenomenon and its robustness
 270 cannot be attributed to those auxiliary mechanisms. Scaling Loop Alignment to deeper networks,
 271 harder benchmarks, and biologically structured connectivity (Dale’s law, sparsity, spiking neurons)
 272 is the natural follow-up. Also, our experiments use a deliberately minimal 3-layer network as a
 273 clean proof of principle. Whether Loop Alignment persists when FF and FB pathways are updated
 274 on different timescales as cortical anatomy suggests is an open question, as is a complete theoretical
 275 treatment of the alignment improvement in fully plastic circuit sharing. Furthermore, our analysis
 276 and experiments focus on networks of 2–4 layers; recent work shows that PC training in deeper
 277 architectures requires additional mechanisms to balance error propagation [Qi et al., 2025], and
 278 whether loop alignment remains effective in that regime is an open question.

279 7 Conclusion

280 Loop Alignment shows that the weight transport problem, long treated as a fundamental obstacle to
 281 biologically plausible learning, is dissolved by the dynamics that make predictive coding distinctive.
 282 The inference loop is not only a mechanism for minimizing prediction error but also an implicit
 283 mechanism to achieve weight symmetry. This mechanism survives under shared and continuously
 284 changing circuits, and tolerates the synaptic instability present in real neural circuits.

References

- A. M. Bastos, W. M. Usrey, R. A. Adams, G. R. Mangun, P. Fries, and K. J. Friston. Canonical microcircuits for predictive coding. *Neuron*, 76(4):695–711, 2012.
- K. Friston. A theory of cortical responses. *Philosophical Transactions of the Royal Society B*, 360(1456):815–836, 2005.
- K. Friston. The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11(2):127–138, 2010.
- K. Friston and S. Kiebel. Predictive coding under the free-energy principle. *Philosophical Transactions of the Royal Society B*, 364(1521):1211–1221, 2009.
- W. Gerstner, W. M. Kistler, R. Naud, and L. Paninski. *Neuronal Dynamics: From Single Neurons to Networks and Models of Cognition*. Cambridge University Press, 2014.
- J. Guerguiev, T. P. Lillicrap, and B. A. Richards. Towards deep learning with segregated dendrites. *eLife*, 6:e22901, 2017.
- G. Hinton. The forward-forward algorithm: Some preliminary investigations. In *arXiv preprint arXiv:2212.13345*, 2022.
- G. E. Hinton. Training products of experts by minimizing contrastive divergence. *Neural Computation*, 14(8):1771–1800, 2002.
- A. Laborieux and F. Zenke. Improving equilibrium propagation without weight symmetry through Jacobian homeostasis. In *International Conference on Learning Representations*, 2024.
- D.-H. Lee, S. Zhang, A. Fischer, and Y. Bengio. Difference target propagation. In *Machine Learning and Knowledge Discovery in Databases*, pages 498–515, 2015.
- Q. Liao, J. Leibo, and T. Poggio. How important is weight symmetry in backpropagation? In *AAAI Conference on Artificial Intelligence*, 2016.
- T. P. Lillicrap, D. Cownden, D. B. Tweed, and C. J. Akerman. Random synaptic feedback weights support error backpropagation for deep learning. *Nature Communications*, 7:13276, 2016.
- B. Millidge, A. Seth, and C. L. Buckley. Predictive coding: a theoretical and experimental review. *arXiv preprint arXiv:2107.12979*, 2022.
- A. Nøkland. Direct feedback alignment provides learning in deep neural networks. In *Advances in Neural Information Processing Systems*, volume 29, 2016.
- C. Qi, M. Forasassi, T. Lukasiewicz, and T. Salvatori. Towards the training of deeper predictive coding neural networks. *arXiv preprint arXiv:2506.23800*, 2025.
- R. P. Rao and D. H. Ballard. Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2(1):79–87, 1999.
- M. Rigotti, O. Barak, M. R. Warden, X.-J. Wang, N. D. Daw, E. K. Miller, and S. Fusi. The importance of mixed selectivity in complex cognitive tasks. *Nature*, 497(7451):585–590, 2013.
- D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. In *Nature*, volume 323, pages 533–536, 1986.
- J. Sacramento, R. P. Costa, Y. Bengio, and W. Senn. Dendritic cortical microcircuits approximate the backpropagation algorithm. *Advances in Neural Information Processing Systems*, 31, 2018.
- B. Scellier and Y. Bengio. Equilibrium propagation: Bridging the gap between energy-based models and backpropagation. *Frontiers in Computational Neuroscience*, 11:24, 2017.
- A. M. Thomson and A. P. Bannister. Interlaminar connections in the neocortex. *Cerebral Cortex*, 13(1):5–14, 2003.
- J. C. Whittington and R. Bogacz. An approximation of the error backpropagation algorithm in a predictive coding network with local Hebbian synaptic plasticity. *Neural Computation*, 29(5):1229–1262, 2017.

331 A Hyperparameters

Table 1: Hyperparameters shared across all models and experiments.

Parameter	Value	Notes
Network architecture	784–100–10	Representation neurons
Error neuron sizes	100–10	
Inference loops (main)	50	Varied to 1/3/5/10 in Sec. 5.2
Inference learning rate η	5×10^{-2}	
FF weight learning rate α	1×10^{-2}	
FB weight learning rate (PC-DH)	1×10^{-2}	Equal to FF rate
Mini-batch size	20	Single-task only; 1 for circuit sharing
Weight initialization	Xavier normal	Mean 0
Training steps (main)	10^6	50 sessions
Training steps (circuit/robustness)	3×10^6	10 sessions
MNIST/Fashion-MNIST normalization	[0, 1]	Pixel values
Teacher signal	One-hot, clamped	Top-layer representation

332 B Cosine similarity computation

333 The FF weight W_i^{FF} has shape $[n, m]$ and the FB weight W_i^{FB} has shape $[m, n]$. To compare
 334 element-wise, we transpose W_i^{FF} to shape $[m, n]$ and flatten both matrices to vectors of length
 335 $m \times n$. Cosine similarity is computed as $\cos(W_i^{\text{FF}\top}, W_i^{\text{FB}}) = \langle W_i^{\text{FF}\top}, W_i^{\text{FB}} \rangle / (\|W_i^{\text{FF}}\| \|W_i^{\text{FB}}\|)$.
 336 This equals 1 when $W^{\text{FF}} = W_i^{\text{FB}\top}$ up to a positive scalar factor, 0 when orthogonal, and -1 when
 337 anti-aligned.

338 C Statistical tests

339 McNemar’s test was used to compare classification performance between model pairs on the same
 340 test set. For each test sample, we recorded whether each model predicted correctly, constructed a
 341 2×2 contingency table from discordant pairs, and computed the McNemar χ^2 statistic. Significance
 342 level: $\alpha = 0.05$.

343 D Alignment drift: complete derivation

344 This section gives the complete derivation of the alignment drift term, complementing the sketch in
 345 Section 4.

346 **Setting and notation.** At layer i , let $\mathbf{x}_i \in \mathbb{R}^n$ and $\boldsymbol{\varepsilon}_i \in \mathbb{R}^m$ be the representation and error vectors
 347 (row vectors), $W_i^{\text{FF}} \in \mathbb{R}^{n \times m}$ the FF weight, and $B_i \in \mathbb{R}^{m \times n}$ the fixed random FB matrix. The
 348 prediction error and inference update read

$$\boldsymbol{\varepsilon}_i = \mathbf{x}_{i+1} - \mathbf{x}_i W_i^{\text{FF}}, \quad (20)$$

$$\tilde{\mathbf{x}}_i = \mathbf{x}_i - \eta \boldsymbol{\varepsilon}_{i-1} + \eta \boldsymbol{\varepsilon}_i B_i. \quad (21)$$

349 After L inference loops (with $\mathbf{x}_{i+1}$ and $\mathbf{x}_{i-1}$ held fixed), the FF weight is updated using the final $\tilde{\mathbf{x}}_i$
 350 and $\boldsymbol{\varepsilon}_i$:

$$\Delta W_i^{\text{FF}} = \alpha \tilde{\mathbf{x}}_i^\top \boldsymbol{\varepsilon}_i. \quad (22)$$

351 **Substitution and expansion.** Substituting (21) into (22):

$$\begin{aligned} \Delta W_i^{\text{FF}} &= \alpha (\mathbf{x}_i - \eta \boldsymbol{\varepsilon}_{i-1} + \eta \boldsymbol{\varepsilon}_i B_i)^\top \boldsymbol{\varepsilon}_i \\ &= \alpha \mathbf{x}_i^\top \boldsymbol{\varepsilon}_i - \alpha \eta \boldsymbol{\varepsilon}_{i-1}^\top \boldsymbol{\varepsilon}_i + \alpha \eta (\boldsymbol{\varepsilon}_i B_i)^\top \boldsymbol{\varepsilon}_i. \end{aligned} \quad (23)$$

352 **Expected drift under the whiteness approximation.** The alignment drift contains the error auto-
 353 correlation $\epsilon_i^\top \epsilon_i$. For a single sample this is a rank-1 outer product with prominent off-diagonal
 354 structure. Averaged over a mini-batch, however, the batch-mean auto-correlation is diagonally dom-
 355 inant throughout training (Fig. 2D). We therefore make the approximation

$$\mathbb{E}[\epsilon_i^\top \epsilon_i] \approx \sigma_\epsilon^2 I, \quad (24)$$

356 where σ_ϵ^2 is the mean squared error magnitude. This *whiteness approximation* is a batch-statistical
 357 property that holds from the first training step, not a consequence of learning (Sec. 5.2; Fig. 2D).

358 Taking the expectation of (16) and applying (24):

$$\mathbb{E}[\Delta W_i^{\text{FF}}] = \alpha \mathbb{E}[\mathbf{x}_i^\top \epsilon_i] - \alpha \eta \mathbb{E}[\epsilon_{i-1}^\top \epsilon_i] + \alpha \eta \sigma_\epsilon^2 B_i^\top. \quad (25)$$

359 The third term is a persistent drift in the direction of $B_i^\top$: at every training step, regardless of the
 360 current value of W_i^{FF} , the inference loop nudges W_i^{FF} toward the transpose of the fixed random
 361 matrix B_i . This is the mechanism of Loop Alignment in PC-RFB.

362 **PC-DH as the $B_i = W_i^{\text{FB}}$ case.** In PC-DH the FB matrix is not fixed; it plays the role of B_i at each
 363 step. The drift term becomes $\alpha \eta \sigma_\epsilon^2 (W_i^{\text{FB}})^\top$, nudging W_i^{FF} toward $(W_i^{\text{FB}})^\top$. Simultaneously, the
 364 symmetric Hebbian rule (Eq. 6) nudges W_i^{FB} toward $(W_i^{\text{FF}})^\top$. Loop Alignment is therefore driven
 365 from *both* directions in PC-DH, which is consistent with the stronger alignment (cosine $\rightarrow 1.0$)
 366 relative to PC-RFB (cosine $\rightarrow 0.9$).

367 E Circuit sharing: relay structure and update rules

368 **Relay matrices.** The shared feedback pathway is parameterized by two relay matrices, $\mathbf{U} \in$
 369 $\mathbb{R}^{m \times n}$ and $\mathbf{V} \in \mathbb{R}^{m \times n}$, where n and m are the representation and error neuron counts of the shared
 370 layer. Both matrices are initialized Xavier-normal and are common to the feedback paths of both
 371 networks. The two effective FB weights traverse the closed loop in opposite directions (Eq. 17):

$$W_i^{\text{FB, Net 1}} = \mathbf{U} W_i^{\text{Net 2}} \mathbf{V}, \quad W_i^{\text{FB, Net 2}} = \mathbf{V} W_i^{\text{Net 1}} \mathbf{U}. \quad (26)$$

372 **Alternating training with one network active per step.** Training proceeds in strict alternation,
 373 with no mini-batching. At every step exactly one of $\{\text{Net 1}, \text{Net 2}\}$ is the *active* network; the other
 374 serves only as a passive relay along the shared pathway. We describe the active-Net 1 step; the
 375 active-Net 2 step is symmetric (interchange labels, swap $\mathbf{U} \leftrightarrow \mathbf{V}$ in the loop-traversal direction).

376 **Net 1-active step: inference.** A single image is sampled and clamped as Net 1’s bottom and
 377 top representations. All weights are held fixed during inference. Net 1’s activities are updated for
 378 $L_{\text{inf}} = 50$ loops by

$$\Delta \mathbf{x}_i^{\text{Net 1}} = \eta (-\epsilon_{i-1}^{\text{Net 1}} + \epsilon_i^{\text{Net 1}} \mathbf{U} W_i^{\text{Net 2}} \mathbf{V}). \quad (27)$$

379 Net 2 is not driven by any external input during this step, so its shared-layer variables are determined
 380 by Net 1’s signal traversing the loop:

$$\mathbf{x}_i^{\text{Net 2}} = \epsilon_i^{\text{Net 1}} \mathbf{U}, \quad \epsilon_i^{\text{Net 2}} = \epsilon_i^{\text{Net 1}} \mathbf{U} W_i^{\text{Net 2}}. \quad (28)$$

381 These are not independent state variables of Net 2; they are the post- $\mathbf{U}$ and post- $W_i^{\text{Net 2}}$ values of
 382 Net 1’s signal, recomputed at every inference loop. After 50 loops, we read out the converged Net 1
 383 activities together with the corresponding final relay values from Eq. (28).

384 **Net 1-active step: weight updates.** Only the weights belonging to the active network’s path are
 385 updated. In the partial-plasticity setting (Figs. 3B–D), $\mathbf{U}$ and $\mathbf{V}$ are fixed and only $W_i^{\text{Net 1}}$ is up-
 386 dated:

$$\Delta W_i^{\text{Net 1}} = \alpha (\mathbf{x}_i^{\text{Net 1}})^\top \epsilon_i^{\text{Net 1}}. \quad (29)$$

387 In the fully plastic setting (Figs. 3E–G), $\mathbf{U}$ and $\mathbf{V}$ additionally receive local Hebbian updates at their
 388 respective synapses, using the active network’s variables and the relay values from Eq. (28):

$$\Delta \mathbf{U}^{(\text{Net 1-active})} = \alpha (\epsilon_i^{\text{Net 1}})^\top \mathbf{x}_i^{\text{Net 2}} = \alpha (\epsilon_i^{\text{Net 1}})^\top (\epsilon_i^{\text{Net 1}} \mathbf{U}), \quad (30)$$

$$\Delta \mathbf{V}^{(\text{Net 1-active})} = \alpha (\epsilon_i^{\text{Net 2}})^\top \mathbf{x}_i^{\text{Net 1}} = \alpha (\epsilon_i^{\text{Net 1}} \mathbf{U} W_i^{\text{Net 2}})^\top \mathbf{x}_i^{\text{Net 1}}. \quad (31)$$

Equation (30) is the local Hebbian product at $\mathbf{U}$'s input/output synapses: pre-synaptic activity $\varepsilon_i^{\text{Net } 1}$ entering $\mathbf{U}$, post-synaptic activity $\mathbf{x}_i^{\text{Net } 2} = \varepsilon_i^{\text{Net } 1} \mathbf{U}$ exiting it. Equation (31) is the analogous product at $\mathbf{V}$: pre-synaptic activity $\varepsilon_i^{\text{Net } 2} = \varepsilon_i^{\text{Net } 1} \mathbf{U} W_i^{\text{Net } 2}$ entering $\mathbf{V}$, post-synaptic representation $\mathbf{x}_i^{\text{Net } 1}$ where the loop closes. The partner network's FF weight $W_i^{\text{Net } 2}$ is *not* updated on this step.

Net 2-active step. The next step interchanges roles. A single image is clamped to Net 2; Net 2's activities update via $\Delta \mathbf{x}_i^{\text{Net } 2} = \eta(-\varepsilon_{i-1}^{\text{Net } 2} + \varepsilon_i^{\text{Net } 2} \mathbf{V} W_i^{\text{Net } 1} \mathbf{U})$, and Net 1's shared-layer variables become the relay values $\mathbf{x}_i^{\text{Net } 1} = \varepsilon_i^{\text{Net } 2} \mathbf{V}$, $\varepsilon_i^{\text{Net } 1} = \varepsilon_i^{\text{Net } 2} \mathbf{V} W_i^{\text{Net } 1}$. The weight updates are the symmetric counterparts of Eqs. (29)–(31), with Net 1 and Net 2 indices swapped and the $\mathbf{U} \leftrightarrow \mathbf{V}$ traversal order reversed accordingly. The two data pools (MNIST and Fashion-MNIST) are shuffled with independent random seeds. The learning rate for all updates is α (Table 1).

F Algorithm descriptions

All models share the same inference loop structure; they differ only in the FB pathway and which weights are updated. Within each mini-batch of size 20, samples are processed *sequentially*: inference and weight update for sample k complete before sample $k+1$ begins. Algorithms 1–3 describe single-task learning; Algorithms 4–5 describe one alternating step of each circuit-sharing condition. For all algorithms, layer indices run over the non-clamped representation layers ($i = 1, \dots, L-1$ for an L -layer network).

Algorithm 1 Baseline PC (one sample)

Require: input $\mathbf{x}_0$, label $\mathbf{x}_L$, weights W^{FF} , learning rates η, α

- 1: **Clamp** $\mathbf{x}_0 \leftarrow \text{input}$, $\mathbf{x}_L \leftarrow \text{label}$
- 2: Initialize $\mathbf{x}_i \leftarrow \mathbf{0}$ for $i = 1, \dots, L-1$
- 3: **for** $\ell = 1$ to L_{inf} **do** ▷ inference loops ($L_{\text{inf}} = 50$)
- 4: **for** $i = 0$ to $L-1$ **do**
- 5: $\varepsilon_i \leftarrow \mathbf{x}_{i+1} - \mathbf{x}_i W_i^{\text{FF}}$
- 6: **end for**
- 7: **for** $i = 1$ to $L-1$ **do**
- 8: $\mathbf{x}_i \leftarrow \mathbf{x}_i + \eta(-\varepsilon_{i-1} + \varepsilon_i (W_i^{\text{FF}})^\top)$
- 9: **end for**
- 10: **end for**
- 11: **for** $i = 0$ to $L-1$ **do** ▷ weight update
- 12: $\varepsilon_i \leftarrow \mathbf{x}_{i+1} - \mathbf{x}_i W_i^{\text{FF}}$
- 13: $W_i^{\text{FF}} \leftarrow W_i^{\text{FF}} + \alpha \mathbf{x}_i^\top \varepsilon_i$
- 14: **end for**

Algorithm 2 PC-DH (one sample)

Require: input $\mathbf{x}_0$, label $\mathbf{x}_L$, weights W^{FF} , W^{FB} , learning rates η, α

- 1: **Clamp** $\mathbf{x}_0, \mathbf{x}_L$; initialize $\mathbf{x}_i \leftarrow \mathbf{0}$
- 2: **for** $\ell = 1$ to L_{inf} **do**
- 3: **for** $i = 0$ to $L-1$ **do**
- 4: $\varepsilon_i \leftarrow \mathbf{x}_{i+1} - \mathbf{x}_i W_i^{\text{FF}}$
- 5: **end for**
- 6: **for** $i = 1$ to $L-1$ **do**
- 7: $\mathbf{x}_i \leftarrow \mathbf{x}_i + \eta(-\varepsilon_{i-1} + \varepsilon_i W_i^{\text{FB}})$
- 8: **end for**
- 9: **end for**
- 10: $\varepsilon_i \leftarrow \mathbf{x}_{i+1} - \mathbf{x}_i W_i^{\text{FF}}$ for all i
- 11: **for** $i = 0$ to $L-1$ **do** ▷ independent Hebbian updates
- 12: $W_i^{\text{FF}} \leftarrow W_i^{\text{FF}} + \alpha \mathbf{x}_i^\top \varepsilon_i$
- 13: $W_i^{\text{FB}} \leftarrow W_i^{\text{FB}} + \alpha \varepsilon_i^\top \mathbf{x}_i$
- 14: **end for**

Algorithm 3 PC-RFB (one sample)

Require: input $\mathbf{x}_0$, label $\mathbf{x}_L$, FF weight W^{FF} , fixed random B (never updated)

- 1: **Clamp** $\mathbf{x}_0, \mathbf{x}_L$; initialize $\mathbf{x}_i \leftarrow \mathbf{0}$
- 2: **for** $\ell = 1$ to L_{inf} **do**
- 3: **for** $i = 0$ to $L - 1$ **do**
- 4: $\varepsilon_i \leftarrow \mathbf{x}_{i+1} - \mathbf{x}_i W_i^{\text{FF}}$
- 5: **end for**
- 6: **for** $i = 1$ to $L - 1$ **do**
- 7: $\mathbf{x}_i \leftarrow \mathbf{x}_i + \eta(-\varepsilon_{i-1} + \varepsilon_i B_i)$
- 8: **end for**
- 9: **end for**
- 10: $\varepsilon_i \leftarrow \mathbf{x}_{i+1} - \mathbf{x}_i W_i^{\text{FF}}$ for all i
- 11: $W_i^{\text{FF}} \leftarrow W_i^{\text{FF}} + \alpha \mathbf{x}_i^\top \varepsilon_i$ $\triangleright B_i$ unchanged

Algorithm 4 Circuit sharing, partial plasticity — one alternating pair of steps

Require: FF weights $W^{\text{Net } 1}, W^{\text{Net } 2}$; fixed relays $\mathbf{U}, \mathbf{V}$; data pools $\mathcal{D}^1$ (MNIST), $\mathcal{D}^2$ (Fashion-MNIST)

// Net 1-active step (Net 2 is passive relay)

- 1: Sample $s^{\text{Net } 1} \sim \mathcal{D}^1$
- 2: **Clamp** $\mathbf{x}_0^{\text{Net } 1}, \mathbf{x}_L^{\text{Net } 1}$ from $s^{\text{Net } 1}$; initialize $\mathbf{x}_i^{\text{Net } 1} \leftarrow \mathbf{0}$
- 3: **for** $\ell = 1$ to L_{inf} **do** $\triangleright$ Net 1 inference, Net 2 as relay
- 4: **for** $i = 0$ to $L - 1$ **do**
- 5: $\varepsilon_i^{\text{Net } 1} \leftarrow \mathbf{x}_{i+1}^{\text{Net } 1} - \mathbf{x}_i^{\text{Net } 1} W_i^{\text{Net } 1}$
- 6: **end for**
- 7: $\mathbf{x}_i^{\text{Net } 2} \leftarrow \varepsilon_i^{\text{Net } 1} \mathbf{U}, \varepsilon_i^{\text{Net } 2} \leftarrow \mathbf{x}_i^{\text{Net } 2} W_i^{\text{Net } 2}$ $\triangleright$ relay values, Eq. (28)
- 8: **for** $i = 1$ to $L - 1$ **do**
- 9: $\mathbf{x}_i^{\text{Net } 1} \leftarrow \mathbf{x}_i^{\text{Net } 1} + \eta(-\varepsilon_{i-1}^{\text{Net } 1} + \varepsilon_i^{\text{Net } 1} \mathbf{U} W_i^{\text{Net } 2} \mathbf{V})$
- 10: **end for**
- 11: **end for**
- 12: $\varepsilon_i^{\text{Net } 1} \leftarrow \mathbf{x}_{i+1}^{\text{Net } 1} - \mathbf{x}_i^{\text{Net } 1} W_i^{\text{Net } 1}$ for all i
- 13: $W_i^{\text{Net } 1} \leftarrow W_i^{\text{Net } 1} + \alpha (\mathbf{x}_i^{\text{Net } 1})^\top \varepsilon_i^{\text{Net } 1}$ $\triangleright \mathbf{U}, \mathbf{V}, W^{\text{Net } 2}$ unchanged

// Net 2-active step (Net 1 is passive relay)

- 14: Sample $s^{\text{Net } 2} \sim \mathcal{D}^2$
- 15: **Clamp** $\mathbf{x}_0^{\text{Net } 2}, \mathbf{x}_L^{\text{Net } 2}$ from $s^{\text{Net } 2}$; initialize $\mathbf{x}_i^{\text{Net } 2} \leftarrow \mathbf{0}$
- 16: **for** $\ell = 1$ to L_{inf} **do** $\triangleright$ Net 2 inference, Net 1 as relay
- 17: **for** $i = 0$ to $L - 1$ **do**
- 18: $\varepsilon_i^{\text{Net } 2} \leftarrow \mathbf{x}_{i+1}^{\text{Net } 2} - \mathbf{x}_i^{\text{Net } 2} W_i^{\text{Net } 2}$
- 19: **end for**
- 20: $\mathbf{x}_i^{\text{Net } 1} \leftarrow \varepsilon_i^{\text{Net } 2} \mathbf{V}, \varepsilon_i^{\text{Net } 1} \leftarrow \mathbf{x}_i^{\text{Net } 1} W_i^{\text{Net } 1}$ $\triangleright$ relay values
- 21: **for** $i = 1$ to $L - 1$ **do**
- 22: $\mathbf{x}_i^{\text{Net } 2} \leftarrow \mathbf{x}_i^{\text{Net } 2} + \eta(-\varepsilon_{i-1}^{\text{Net } 2} + \varepsilon_i^{\text{Net } 2} \mathbf{V} W_i^{\text{Net } 1} \mathbf{U})$
- 23: **end for**
- 24: **end for**
- 25: $\varepsilon_i^{\text{Net } 2} \leftarrow \mathbf{x}_{i+1}^{\text{Net } 2} - \mathbf{x}_i^{\text{Net } 2} W_i^{\text{Net } 2}$ for all i
- 26: $W_i^{\text{Net } 2} \leftarrow W_i^{\text{Net } 2} + \alpha (\mathbf{x}_i^{\text{Net } 2})^\top \varepsilon_i^{\text{Net } 2}$ $\triangleright \mathbf{U}, \mathbf{V}, W^{\text{Net } 1}$ unchanged

Algorithm 5 Circuit sharing, full plasticity — one alternating pair of steps

Require: FF weights $W^{\text{Net } 1}$, $W^{\text{Net } 2}$; learnable relays $\mathbf{U}$, $\mathbf{V}$; data pools $\mathcal{D}^1$ (MNIST), $\mathcal{D}^2$ (Fashion-MNIST)

// Net 1-active step

- 1: Sample $s^{\text{Net } 1} \sim \mathcal{D}^1$; **Clamp** $\mathbf{x}_0^{\text{Net } 1}, \mathbf{x}_L^{\text{Net } 1}$; initialize $\mathbf{x}_i^{\text{Net } 1} \leftarrow \mathbf{0}$
- 2: **for** $\ell = 1$ to L_{inf} **do** ▷ Net 1 inference, Net 2 as relay
- 3: **for** $i = 0$ to $L - 1$ **do**
- 4: $\epsilon_i^{\text{Net } 1} \leftarrow \mathbf{x}_{i+1}^{\text{Net } 1} - \mathbf{x}_i^{\text{Net } 1} W_i^{\text{Net } 1}$
- 5: **end for**
- 6: $\mathbf{x}_i^{\text{Net } 2} \leftarrow \epsilon_i^{\text{Net } 1} \mathbf{U}$, $\epsilon_i^{\text{Net } 2} \leftarrow \mathbf{x}_i^{\text{Net } 2} W_i^{\text{Net } 2}$ ▷ relay values
- 7: **for** $i = 1$ to $L - 1$ **do**
- 8: $\mathbf{x}_i^{\text{Net } 1} \leftarrow \mathbf{x}_i^{\text{Net } 1} + \eta(-\epsilon_{i-1}^{\text{Net } 1} + \epsilon_i^{\text{Net } 1} \mathbf{U} W_i^{\text{Net } 2} \mathbf{V})$
- 9: **end for**
- 10: **end for**
- 11: $\epsilon_i^{\text{Net } 1} \leftarrow \mathbf{x}_{i+1}^{\text{Net } 1} - \mathbf{x}_i^{\text{Net } 1} W_i^{\text{Net } 1}$ for all i ; refresh $\mathbf{x}_i^{\text{Net } 2}, \epsilon_i^{\text{Net } 2}$ via Eq. (28)
- 12: $W_i^{\text{Net } 1} \leftarrow W_i^{\text{Net } 1} + \alpha (\mathbf{x}_i^{\text{Net } 1})^\top \epsilon_i^{\text{Net } 1}$
- 13: $\mathbf{U} \leftarrow \mathbf{U} + \alpha (\epsilon_i^{\text{Net } 1})^\top \mathbf{x}_i^{\text{Net } 2}$ ▷ Eq. (30)
- 14: $\mathbf{V} \leftarrow \mathbf{V} + \alpha (\epsilon_i^{\text{Net } 2})^\top \mathbf{x}_i^{\text{Net } 1}$ ▷ Eq. (31); $W^{\text{Net } 2}$ unchanged

// Net 2-active step (symmetric)

- 15: Sample $s^{\text{Net } 2} \sim \mathcal{D}^2$; **Clamp** $\mathbf{x}_0^{\text{Net } 2}, \mathbf{x}_L^{\text{Net } 2}$; initialize $\mathbf{x}_i^{\text{Net } 2} \leftarrow \mathbf{0}$
- 16: **for** $\ell = 1$ to L_{inf} **do** ▷ Net 2 inference, Net 1 as relay
- 17: **for** $i = 0$ to $L - 1$ **do**
- 18: $\epsilon_i^{\text{Net } 2} \leftarrow \mathbf{x}_{i+1}^{\text{Net } 2} - \mathbf{x}_i^{\text{Net } 2} W_i^{\text{Net } 2}$
- 19: **end for**
- 20: $\mathbf{x}_i^{\text{Net } 1} \leftarrow \epsilon_i^{\text{Net } 2} \mathbf{V}$, $\epsilon_i^{\text{Net } 1} \leftarrow \mathbf{x}_i^{\text{Net } 1} W_i^{\text{Net } 1}$ ▷ relay values
- 21: **for** $i = 1$ to $L - 1$ **do**
- 22: $\mathbf{x}_i^{\text{Net } 2} \leftarrow \mathbf{x}_i^{\text{Net } 2} + \eta(-\epsilon_{i-1}^{\text{Net } 2} + \epsilon_i^{\text{Net } 2} \mathbf{V} W_i^{\text{Net } 1} \mathbf{U})$
- 23: **end for**
- 24: **end for**
- 25: $\epsilon_i^{\text{Net } 2} \leftarrow \mathbf{x}_{i+1}^{\text{Net } 2} - \mathbf{x}_i^{\text{Net } 2} W_i^{\text{Net } 2}$ for all i ; refresh $\mathbf{x}_i^{\text{Net } 1}, \epsilon_i^{\text{Net } 1}$
- 26: $W_i^{\text{Net } 2} \leftarrow W_i^{\text{Net } 2} + \alpha (\mathbf{x}_i^{\text{Net } 2})^\top \epsilon_i^{\text{Net } 2}$
- 27: $\mathbf{U} \leftarrow \mathbf{U} + \alpha (\epsilon_i^{\text{Net } 2})^\top \mathbf{x}_i^{\text{Net } 1}$ ▷ Net 2-active $\Delta \mathbf{U}$
- 28: $\mathbf{V} \leftarrow \mathbf{V} + \alpha (\epsilon_i^{\text{Net } 1})^\top \mathbf{x}_i^{\text{Net } 2}$ ▷ Net 2-active $\Delta \mathbf{V}$; $W^{\text{Net } 1}$ unchanged

406 G Computational resources

407 All experiments were run on a single CPU (no GPU acceleration). Main single-task experiments
 408 (10^6 steps, 50 independent sessions per model) required approximately 10 minutes per session, or
 409 roughly 25 CPU-hours per model variant. Circuit-sharing and robustness experiments (3×10^6 steps,
 410 10 sessions) required approximately 30 minutes per session. The full robustness sweep (7 noise
 411 levels, 10 sessions each) required approximately 35 CPU-hours in total. No hyperparameter search
 412 was performed; all values listed in Table 1 were fixed a priori.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: All three main claims — that both models match baseline PC accuracy, that Loop Alignment holds exactly at every training step, and that alignment survives and strengthens under multi-task circuit sharing — are directly supported by the corresponding experimental and theoretical results in the paper, and no claim exceeds the scope of the minimal 3-layer linear network on MNIST and Fashion-MNIST explicitly described.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: We included the dedicated "Limitations and future directions" section in Discussion.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: All assumptions are stated explicitly: Section 4.1 requires no approximation and proves the transpose identity pointwise (Eq. 5); Section 4.2 states the whiteness approximation explicitly, validates it empirically (Fig. 2D), and derives the drift term by elementary matrix algebra (Eq. 7–8).

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Hyperparameter, network architecture, optimizer, mini-batch size, MNIST/Fashion-MNIST pre-processing, number of inference loops, number of sessions, and statistical test are fully specified in Section 5 Setup, Table 1, and Appendix C/F.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

520 (d) We recognize that reproducibility may be tricky in some cases, in which case au-
521 thors are welcome to describe the particular way they provide for reproducibility.
522 In the case of closed-source models, it may be that access to the model is limited in
523 some way (e.g., to registered users), but it should be possible for other researchers
524 to have some path to reproducing or verifying the results.

525 5. Open access to data and code

526 Question: Does the paper provide open access to the data and code, with sufficient instruc-
527 tions to faithfully reproduce the main experimental results, as described in supplemental
528 material?

529 Answer: [Yes]

530 Justification: Code is deposited at [[https://anonymous.4open.science/r/Loop-Alignment-](https://anonymous.4open.science/r/Loop-Alignment-E46C/)
531 [E46C/](https://anonymous.4open.science/r/Loop-Alignment-E46C/)] and implements all three model variants (PC baseline, PC-DH, PC-RFB) and the
532 circuit-sharing experiment. A README provides installation instructions and scripts to
533 reproduce all figures in the paper.

534 Guidelines:

- 535 • The answer [N/A] means that paper does not include experiments requiring code.
- 536 • Please see the NeurIPS code and data submission guidelines ([https://neurips.cc/](https://neurips.cc/public/guides/CodeSubmissionPolicy)
537 [public/guides/CodeSubmissionPolicy](https://neurips.cc/public/guides/CodeSubmissionPolicy)) for more details.
- 538 • While we encourage the release of code and data, we understand that this might not
539 be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not
540 including code, unless this is central to the contribution (e.g., for a new open-source
541 benchmark).
- 542 • The instructions should contain the exact command and environment needed to run to
543 reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- 544 • The authors should provide instructions on data access and preparation, including how
545 to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- 546 • The authors should provide scripts to reproduce all experimental results for the new
547 proposed method and baselines. If only a subset of experiments are reproducible, they
548 should state which ones are omitted from the script and why.
- 549 • At submission time, to preserve anonymity, the authors should release anonymized
550 versions (if applicable).
- 551 • Providing as much information as possible in supplemental material (appended to the
552 paper) is recommended, but including URLs to data and code is permitted.
- 553

554 6. Experimental setting/details

555 Question: Does the paper specify all the training and test details (e.g., data splits, hyperpa-
556 rameters, how they were chosen, type of optimizer) necessary to understand the results?

557 Answer: [Yes]

558 Justification: We explained all model in methods section. We also included the set of
559 hyperparameters used in this study in appendix section.

560 Guidelines:

- 561 • The answer [N/A] means that the paper does not include experiments.
- 562 • The experimental setting should be presented in the core of the paper to a level of
563 detail that is necessary to appreciate the results and make sense of them.
- 564 • The full details can be provided either with the code, in appendix, or as supplemental
565 material.

566 7. Experiment statistical significance

567 Question: Does the paper report error bars suitably and correctly defined or other appropri-
568 ate information about the statistical significance of the experiments?

569 Answer: [Yes]

Justification: All experimental results report mean and standard deviation across 50 independent sessions for main experiments and 10 sessions for circuit-sharing and robustness experiments (shaded regions in all figures). Statistical significance of accuracy differences between models is assessed via McNemar’s test (Appendix C), with all comparisons against baseline yielding $p > 0.05$.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The paper reports information for compute resources in Appendix G . All experiments were run on CPU, with main experiments (10^6 iterations, 50 sessions) taking approximately 10 min and circuit-sharing experiments (3×10^6 iterations, 10 sessions) taking approximately 30 min.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn’t make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

Answer: [Yes]

Justification: This paper presents theoretical and computational neuroscience research involving no human subjects, no sensitive data, and no dual-use risks. All experiments use standard public benchmarks (MNIST and Fashion-MNIST). The work poses no foreseeable negative societal impact.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: This paper presents foundational theoretical and computational neuroscience research with no direct path to deployable systems or applications. It involves only standard public benchmarks (MNIST and Fashion-MNIST), no human subjects, and no sensitive data. Following the NeurIPS guidelines, no societal impact discussion is required for work of this nature.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper releases only training code for a minimal 3-layer linear network on MNIST and Fashion-MNIST. No pretrained models, scraped datasets, or high-risk generative systems are involved, and no safeguards are required.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.

- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The paper uses two standard public benchmarks: MNIST and Fashion-MNIST, both released under the MIT License and accessed via their standard distributions. The PC model follows Whittington and Bogacz [2017] and is cited accordingly. No other external code or pretrained models are use

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The released code is documented with a README providing installation instructions, dependency list, and scripts to reproduce all main figures. The repository is anonymized and available at [<https://anonymous.4open.science/r/Loop-Alignment-E46C/>].

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper involves no crowdsourcing or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper involves no crowdsourcing or research with human subjects and therefore requires no IRB approval.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLMs were used strictly for writing, editing and formatting purposes.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.